

Referência de comandos ADB e Android

Compilado durante o desenvolvimento e depuração do FlutterTap

Eduardo Lopes
29 de julho de 2026

Compilado a partir do desenvolvimento e depuração do módulo FlutterTap (Zygisk). Todo comando aqui foi realmente executado e verificado em aparelho físico — OnePlus 5 (Android 10, Magisk) e Pixel 8a (Android 17, SukiSu Ultra).

1. Conexão e identificação do aparelho

```
adb devices -l
```

Lista os aparelhos conectados com produto, modelo e transport_id. Use sempre antes de qualquer outra coisa: se houver mais de um aparelho plugado, todo comando seguinte precisa de -s.

```
adb -s 3C091JEKB06435 shell ...
```

Direciona o comando para um aparelho específico pelo número de série (a primeira coluna de adb devices). Com dois aparelhos conectados, sem isso o adb recusa o comando.

```
adb get-state
```

Retorna 'device' quando está conectado e pronto. Útil em laço de espera após reiniciar.

```
adb shell getprop ro.build.version.release
```

Versão do Android. Outras propriedades úteis: ro.product.model, ro.build.version.sdk (nível de API), ro.product.cpu.abi (arquitetura).

Atenção: Dica de automação: depois de 'adb reboot', o adb demora a voltar e pode retornar erros como 'device offline' ou 'no devices found'. Faça um laço checando 'adb get-state' e só então 'getprop sys.boot_completed'.

2. Root e a armadilha do redirecionamento

```
adb shell su -c "id"
```

Verifica se o root funciona. A saída mostra também o contexto SELinux: 'u:r:ksu:s0' indica KernelSU/SukiSu, 'u:r:magisk:s0' indica Magisk. Se retornar vazio ou negado, o gerenciador de root não concedeu permissão para o shell do adb.

```
adb shell su -c "cat > /data/adb/arquivo"      # ERRADO
```

Isto falha com 'Permission denied' mesmo com root funcionando. O motivo: o redirecionamento '>' é executado pelo shell EXTERNO (sem root), não pelo su. O su recebe apenas 'cat'.

```
adb shell "su -c 'echo conteudo | tee /data/adb/arquivo'"
```

Forma correta com tee: as aspas simples internas fazem o su receber o comando completo. O tee escreve como root.

```
adb push arq.json /data/local/tmp/arq.json
adb shell su -c "cp /data/local/tmp/arq.json /data/adb/destino.json"
```

Alternativa mais robusta para arquivos maiores ou com formatação: empurra para /data/local/tmp (gravável pelo shell) e copia como root. O cp não depende de redirecionamento.

3. Gerenciamento de módulos

```
adb shell su -c "ksud module list"
```

KernelSU / SukiSu Ultra: lista os módulos em JSON, com id, enabled, versão e description. O campo description costuma trazer status em tempo real (ex.: o Zygisk Next informa se o zygote foi injetado).

```
adb shell su -c "ksud module install /data/local/tmp/modulo.zip"
adb shell su -c "ksud module enable <id>"
adb shell su -c "ksud module disable <id>"
```

Instalar, habilitar e desabilitar módulos. Desabilitar tem efeito imediato no estado, mas só passa a valer de fato após reiniciar.

```
adb shell su -c "magisk -v"
adb shell su -c "magisk --install-module /data/local/tmp/modulo.zip"
```

Equivalentes no Magisk. O 'magisk -v' também revela forks (ex.: 'v27.2-kitsune').

Atenção: A instalação NÃO substitui os arquivos imediatamente: ela grava em /data/adb/modules_update/<id>/ e só move para /data/adb/modules/<id>/ no próximo boot. Se o .so não aparecer no diretório ativo, não conclua que falhou — confira o modules_update.

```
adb shell su -c "ls /data/adb/modules/"  
adb shell su -c "ls /data/adb/modules/<id>/disable"
```

Lista módulos instalados. A presença de um arquivo 'disable' dentro da pasta indica que aquele módulo está desativado — é assim que protetores de bootloop desligam módulos suspeitos.

4. Verificação de integridade (não confie, verifique)

```
adb shell su -c "sha256sum /data/adb/modules/<id>/zygisk/arm64-v8a.so"  
sha256sum dist/stage/zygisk/arm64-v8a.so
```

Compare o checksum do que está no aparelho com o do build local. Esta é a checagem mais valiosa da lista: numa sessão real, um build abortou silenciosamente e o comando de instalação enviou o pacote ANTERIOR — o teste 'passou' validando código velho. Só o checksum revelou.

5. Logs e diagnóstico

```
adb logcat -c
```

Limpa o buffer. Faça isto imediatamente antes da ação que quer observar, para não misturar com histórico.

```
adb shell su -c "logcat -d -s FlutterTap:V"
```

Despeja o buffer filtrando por uma tag específica em nível verbose. O -d despeja e sai (sem -d o comando fica preso seguindo o log). Rodar via su é necessário quando o processo alvo é de outro usuário.

```
adb shell su -c "logcat -d" | grep -iE "fatal|crash|abort|died"
```

Varredura ampla por sinais de crash quando você não sabe a tag. Procure também por 'signal 11 (SIGSEGV)' e 'Process ... has died'.

```
adb shell su -c "dmesg" | head -3
```

Buffer do kernel. Cuidado: ele dá a volta rápido (1024 KB em poucos minutos de uptime). Se a primeira linha tiver timestamp muito maior que zero, os logs de boot já foram sobrescritos e não há como

recuperá-los — use tombstones.

```
adb shell su -c "ls -lt /data/tombstones/ | head"
```

Relatórios nativos de crash, mais recentes primeiro. Cada tombstone traz sinal, mensagem de abort, backtrace e — muito útil — o campo 'Process uptime', que diz QUANDO o processo morreu e ajuda a separar 'falhou ao carregar' de 'morreu em uso'.

6. Processos e estado do sistema

```
adb shell su -c "ps -A" | grep -c "^u0_a"
```

Conta processos de aplicativo de usuário. Sinal de saúde extremamente útil: num sistema saudável são dezenas (120+ num Pixel). Se der 0 ou muito baixo, o zygote não está conseguindo criar processos — indício de crash loop.

```
adb shell su -c "pidof com.android.systemui"
```

Verifica se a interface do sistema está viva. Se o SystemUI estiver morto, a tela fica preta mesmo com o aparelho ligado e respondendo ao adb.

```
adb shell su -c "cat /proc/uptime"  
adb shell su -c "cat /proc/<pid>/stat"
```

Tempo desde o boot e estatísticas do processo. No stat, o campo 22 é o starttime em ticks de clock desde o boot — divida por 'getconf CLK_TCK' (normalmente 100) para saber há quanto tempo o processo existe. Serve para provar que um processo não reiniciou.

```
adb shell su -c "am force-stop <pacote>"
```

Encerra o app completamente. Obrigatório ao testar hooks: um módulo Zygisk só atua em processos NOVOS, então reabrir sem forçar a parada não aplica a configuração nova.

```
adb shell pm list packages | grep -i <parte-do-nome>
```

Descobre o nome exato do pacote. Sempre consulte em vez de supor — nomes reais surpreendem (ex.: 'co.ostorlab.ostorlab_insecure_flutter_app.host', com sufixo .host).

```
adb shell monkey -p <pacote> -c android.intent.category.LAUNCHER 1
```

Abre o app pela activity de launcher sem precisar saber o nome da activity. Mais prático que 'am start -n pacote/.MainActivity' quando você não conhece a estrutura interna.

7. Automação de interface

```
adb shell uiautomator dump /sdcard/ui.xml  
adb pull /sdcard/ui.xml .
```

Despeja a árvore de acessibilidade com as coordenadas REAIS de cada elemento, no atributo `bounds="[x1,y1][x2,y2]"`. Este é o método confiável para saber onde tocar.

Atenção: Lição aprendida na prática: NUNCA calcule coordenadas a partir de um screenshot exibido em escala. Uma imagem de 1080x2400 costuma ser exibida a 900x2000 — usar as coordenadas da exibição erra o alvo por 20%. Sempre use o bounds do uiautomator, ou multiplique pelo fator de escala corretamente. Se um toque não fizer o esperado, tire novo dump em vez de repetir o palpite.

```
adb shell input tap 540 1318  
adb shell input text "admin"  
adb shell input swipe 540 1900 540 700 200
```

Toque, digitação e arraste (o último número é a duração em ms). O input text não aceita espaços diretamente — use %s ou aspas.

```
adb shell input keyevent 61      # TAB  
adb shell input keyevent 66     # ENTER  
adb shell input keyevent 67     # DEL (backspace)  
adb shell input keyevent 111    # ESC  
adb shell input keyevent 123    # MOVE_END  
adb shell input keyevent 224    # WAKEUP (acorda a tela)
```

Códigos de tecla úteis. O TAB é a forma correta de passar de um campo de formulário para o próximo: tocar no campo seguinte falha quando o teclado virtual já está cobrindo a posição dele.

Atenção: Outra armadilha real: para limpar um campo, tocar nele coloca o cursor onde você tocou, e os backspaces só apagam o que está ANTES do cursor. Envie keyevent 123 (MOVE_END) primeiro e depois os backspaces.

```
adb shell screencap -p /sdcard/s.png
adb pull /sdcard/s.png .
```

Captura de tela. Combine com o uiautomator: o dump diz onde as coisas estão, o screenshot mostra o que elas são.

```
adb shell wm size
adb shell wm density
adb shell wm density 340
adb shell wm density reset
```

Resolução física e densidade (DPI). Alterar a densidade muda o layout dos apps e é reversível com 'reset' — útil quando um elemento está inacessível.

```
adb shell cmd overlay list android | grep navbar
```

Revela o modo de navegação: 'threebutton' (barra com três botões) ou 'gestural' (gestos). Importante porque a barra de três botões FICA POR CIMA do conteúdo do app: um botão do app posicionado no rodapé pode ser intocável, e o toque acaba acionando o botão Home.

```
adb shell settings put system accelerometer_rotation 0
adb shell settings put system user_rotation 1
adb shell settings put system user_rotation 0
adb shell settings put system accelerometer_rotation 1
```

Força paisagem (user_rotation 1) e volta ao normal. É obrigatório desligar o accelerometer_rotation primeiro — com a rotação automática ligada, o user_rotation é sobrescrito e volta sozinho para retrato. Girar a tela foi a solução para alcançar um botão que ficava atrás da barra de navegação.

```
adb shell dumpsys power | grep -iE "mWakefulness"
adb shell dumpsys window | grep -iE "isKeyguardShowing|mCurrentFocus"
```

Estado da tela e do bloqueio. 'mWakefulness=Dozing' com 'isKeyguardShowing=true' explica por que o uiautomator retorna uma árvore vazia: o app está atrás da tela de bloqueio. Já 'mCurrentFocus' confirma qual app está realmente em primeiro plano — essencial para saber se o toque foi para o lugar certo.

8. Ambiente: Git Bash no Windows

```
export MSYS2_ARG_CONV_EXCL='*'
```

No Git Bash, o MSYS converte automaticamente argumentos que parecem caminho Unix. Isso corrompe caminhos do Android: `/data/local/tmp` virou `C:/Program Files/Git/data/local/tmp`. Esta variável desliga a conversão.

Atenção: Mas cuidado — e este foi um erro real com consequência: exportar `MSYS2_ARG_CONV_EXCL='*'` desativa a conversão para o shell INTEIRO. Ferramentas nativas do Windows (como o `llvm-strip` do NDK) passam então a receber caminhos MSYS (`/c/Users/...`) que não conseguem abrir, e falham. Prefira aplicar a variável apenas ao comando do `adb`, ou converta explicitamente com `'cygpath -w'` antes de chamar binários do Windows.

```
adb shell su -c "ksud module list" > saida.json
python -c "import json;d=json.load(open('saida.json',encoding='utf-8'))"
```

Descrições de módulos costumam conter emoji. No Windows, o Python quebra com `UnicodeEncodeError` ao imprimir isso no console (cp1252). Abra sempre com `encoding='utf-8'` e, se precisar imprimir, defina `PYTHONIOENCODING=utf-8`.

9. Sequência de teste de um módulo Zygisk

Ordem que se mostrou confiável, com verificação em cada etapa:


```
# 1. instalar e CONFERIR que o binário certo chegou
adb push modulo.zip /data/local/tmp/m.zip
adb shell su -c "ksud module install /data/local/tmp/m.zip"
adb shell su -c "sha256sum /data/adb/modules_update/<id>/zygisk/arm64-v8a.so"

# 2. reiniciar e esperar de verdade
adb reboot
# laço: get-state == device, depois sys.boot_completed == 1

# 3. saúde do sistema ANTES de testar funcionalidade
adb shell su -c "ps -A" | grep -c "^u0_a"
adb shell su -c "pidof com.android.systemui"
adb shell su -c "ls -lt /data/tombstones/ | head -3"
adb shell su -c "ls /data/adb/modules/<id>/disable"

# 4. so entao o teste funcional
adb shell su -c "am force-stop <pacote>"
adb logcat -c
adb shell monkey -p <pacote> -c android.intent.category.LAUNCHER 1
adb shell su -c "logcat -d -s SuaTag:V"
```

Atenção: Por que checar saúde antes de funcionalidade: um módulo defeituoso pode derrubar TODOS os processos de app. Nesse estado o aparelho responde ao adb normalmente, mas está com a tela preta — e você perde tempo investigando 'por que o hook não funcionou' quando o problema real é que nenhum app consegue nem iniciar.

Atenção: Ausência de crash num teste curto não prova nada. Numa sessão real, uma correção pareceu funcionar após alguns minutos de aparelho ocioso; o travamento só apareceu cerca de 40 minutos depois, quando apps passaram a ser abertos de fato. Exercite o caminho real — abrir, trocar e reabrir apps — antes de concluir que está corrigido.